

DevOps Assignment – GCP Project Setup

Objective

Before any service can be deployed, the GCP project must be properly initialised. This phase sets up the foundation that every subsequent task depends on — the project container, billing link, required APIs, and default region.

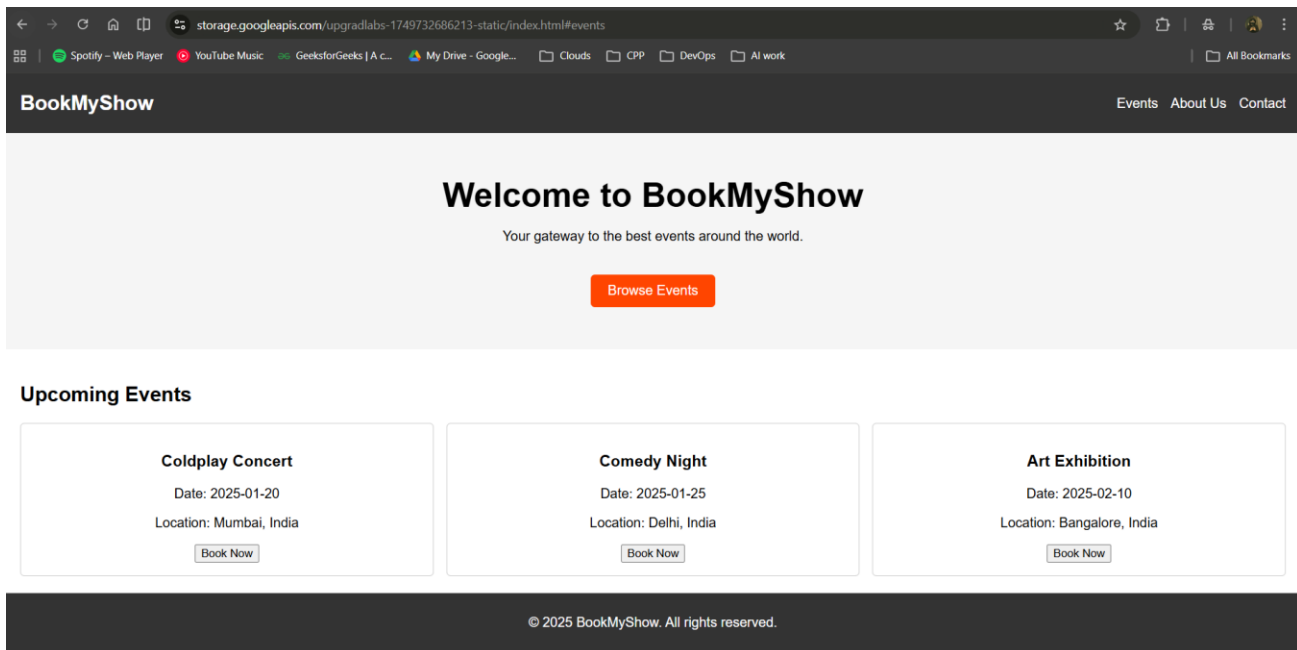
What Was Done

1. Opened Google Cloud Shell from the GCP Console top toolbar.
2. Project: upgradlabs-1749732686213.
3. Linked billing account to the project.
4. Enabled 13 required GCP APIs: Compute Engine, Cloud SQL, Redis, Cloud Run, Cloud Functions, Cloud Build, Cloud Storage, Cloud Monitoring, Cloud Logging, Billing Budgets, Pub/Sub, Cloud Scheduler, Artifact Registry.
5. Set default region to asia-south1 (Mumbai) and zone to asia-south1-a.
6. Created Cloud Storage bucket: upgradlabs-1749732686213-static.
7. Made the bucket publicly readable.
8. Uploaded the three BookMyShow prototype files: index.html, styles.css, script.js.
9. Configured the bucket to serve index.html as the default web page.

Screenshots

```
--- Project ---
Your active configuration is: [cloudshell-22541]
upgradlabs-1749732686213
--- Region ---
Your active configuration is: [cloudshell-22541]
asia-south1
--- Zone ---
Your active configuration is: [cloudshell-22541]
asia-south1-a
--- Bucket files ---
gs://upgradlabs-1749732686213-static/index.html
gs://upgradlabs-1749732686213-static/script.js
gs://upgradlabs-1749732686213-static/styles.css
--- Site status ---
HTTP 200
```

Screenshot: Project details



Screenshot: BookMyShow page

Conclusion

The basis of the GCP project is complete. The Mumbai area is established, all 13 necessary APIs are enabled, and the BookMyShow prototype is operational and available to the public via Cloud Storage.

Task 1 — System Analysis and Architecture Review

Objective

Deploy the BookMyShow application on a single Compute Engine VM to simulate the monolithic baseline architecture. Run traffic simulation tests at 5 load levels to establish baseline performance metrics. Identify bottlenecks in database operations, API response times, and load balancing.

What Was Done

Step 1 — Firewall Rule

Created a firewall rule allowing TCP traffic on ports 80, 5000, and 8089 for all VMs tagged bms-server. This opened the application and Locust UI ports to incoming traffic.

Step 2 — Compute Engine VM

Created a single e2-medium VM named bms-monolith in asia-south1-a. Installed Python, pip and Flask, then launched the BookMyShow API application on port 5000. The app served 5 endpoints: home, events list, seat availability, booking, and health check.

Step 3 — Flask API Endpoints

- GET / — Home page confirmation
- GET /api/events — Returns 3 events (Coldplay, Comedy Night, Art Exhibition)
- GET /api/events/{id}/seats — Returns seat availability for a given event
- POST /api/bookings — Accepts booking request, returns booking ID
- GET /health — Health check endpoint returning status OK

Step 4 — Locust Traffic Simulation

Installed Locust on Cloud Shell and created a test script simulating realistic BookMyShow user behaviour across 4 task types weighted by real usage patterns: browse events (35%), check seat availability (30%), initiate booking (20%), health check (15%). Ran 5 separate tests at increasing user loads with 60 second duration each.

Baseline Performance Results

Concurrent Users	Total Requests	Avg Response	p95 Latency	Max Latency	Failures
50	1,266	232 ms	390 ms	430 ms	0 (0%)
100	2,510	234 ms	390 ms	480 ms	0 (0%)
250	5,432	232 ms	390 ms	550 ms	0 (0%)
500	10,838	238 ms	390 ms	1,249 ms	0 (0%)
1,000	22,566	265 ms	430 ms	1,300 ms	0 (0%)

Bottlenecks Identified

- Single VM = single point of failure — no redundancy or failover.
- Max latency climbed from 430ms to 1,300ms between 50 and 1,000 users — VM CPU saturating.

- POST /api/bookings was the first endpoint to degrade — booking operations are most resource-intensive.
- No load balancing — all traffic hits one server.
- No caching — every request hits the application directly.
- No auto-scaling — cannot add capacity under load.

Screenshots

```
Created [https://www.googleapis.com/compute/v1/projects/upgradlabs-1749732686213/zones/asia-south1-a/instances/bms-monolith].
NAME: bms-monolith
ZONE: asia-south1-a
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.160.0.2
EXTERNAL_IP: 35.200.237.254
STATUS: RUNNING
```

Screenshot: VM Details

Aggregated		1266	0(0.00%)	232	124	434	230	21.25	0.00		
Response time percentiles (approximated)											
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	100% # reqs
POST	/api/bookings	330	380	390	400	420	430	430	430	430	240
GET	/api/events	240	250	260	270	280	280	280	290	290	442
GET	/api/events/1/seats	220	230	240	240	250	250	260	260	260	119
GET	/api/events/2/seats	210	230	230	240	250	250	260	260	260	140
GET	/api/events/3/seats	210	220	230	230	240	240	250	250	260	129
GET	/health	130	140	140	140	140	140	140	170	400	196
Aggregated		230	250	260	270	330	390	410	430	430	1266

sumankumar02304_791d6868_ea38_42@cloudshell:~ (upgradlabs-1749732686213) \$

Aggregated		2510	0(0.00%)	234	124	475	230	42.04	0.00		
Response	time percentiles (approximated)										
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	100% # reqs
POST	/api/bookings	330	370	390	400	420	430	430	440	480	499
GET	/api/events	240	250	260	260	270	280	280	290	300	317
GET	/api/events/1/seats	210	230	240	240	250	250	260	260	260	263
GET	/api/events/2/seats	210	230	240	240	250	250	260	260	260	251
GET	/api/events/3/seats	210	220	230	230	240	250	250	260	260	221
GET	/health	130	140	140	140	140	140	140	150	150	359
Aggregated		230	250	260	270	330	390	420	430	440	2510

sumankumar02304_791d6060_ea30_420cloudshell~ (upgradlabs-1749732606213) \$

Aggregated		10838		0(0.00%)		238		123		1249		230		181.38		0.00	
Response	time percentiles (approximated)																
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs				
POST	/api/bookings	340	370	390	400	420	430	460	470	500	1200	1200	2179				
GET	/api/events	240	260	260	270	280	280	290	290	340	360	360	3804				
GET	/api/events/1/seats	220	230	240	240	250	250	260	270	320	320	320	1052				
GET	/api/events/2/seats	210	230	240	240	250	260	260	270	310	310	310	1107				
GET	/api/events/3/seats	220	230	240	240	250	260	270	290	310	320	320	1071				
GET	/health	140	140	140	140	140	150	160	180	210	210	210	1625				
Aggregated		230	250	270	280	340	390	420	430	480	500	1200	10838				

Aggregated		22566	0(0.00%)	265	125	1278	250	376.77	0.00		
Response time percentiles (approximated)											
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	100% # reqs
POST	/api/bookings	380	410	420	440	460	490	530	560	640	4484
GET	/api/events	260	280	290	290	320	350	390	410	490	7908
GET	/api/events/1/seats	240	250	260	270	300	330	380	410	520	2263
GET	/api/events/2/seats	240	250	260	270	290	320	360	390	450	2226
GET	/api/events/3/seats	240	250	260	270	290	330	380	410	510	2299
GET	/health	150	160	170	180	200	240	280	300	380	3386
Aggregated		250	280	300	320	390	430	460	490	590	22566

Screenshot: Locust test output

Conclusion

The single VM baseline was successfully established. The system handled all 5 load levels with zero failures, but clear performance degradation appeared at higher loads — max latency tripling from 430ms to 1,300ms. These metrics serve as the 'before' benchmark. The absence of auto-scaling, load balancing, caching, and database read replicas are the primary architectural gaps this project addresses in Tasks 2 through 7.

Task 2 — Scalable Architecture Implementation

Objective

Replace the single VM with a Managed Instance Group that can auto-scale based on CPU load. Deploy a Global HTTP Load Balancer with deep health checks to distribute traffic evenly across all healthy instances. Migrate stateless services to Cloud Run using a phased approach.

What Was Done

Step 1 — Instance Template

Created an instance template named `bms-app-template` using `e2-medium` machines running Debian 11. The template includes a startup script that installs Flask and launches the BookMyShow API automatically on every new instance. All instances created from this template are identical and interchangeable.

Step 2 — Managed Instance Group

Created a Managed Instance Group named `bms-instance-group` in `asia-south1-a` starting with 2 instances. Both instances (`bms-instance-group-59g9` and `bms-instance-group-b874`) launched successfully from the template and were confirmed `RUNNING`.

Step 3 — Auto-Scaling Configuration

Configured auto-scaling on the MIG with the following rules: minimum 2 instances always running, maximum 5 instances at peak, scale up when CPU exceeds 60% for 90 seconds, cool-down period of 90 seconds between scaling events. This ensures new instances are added before existing ones reach saturation.

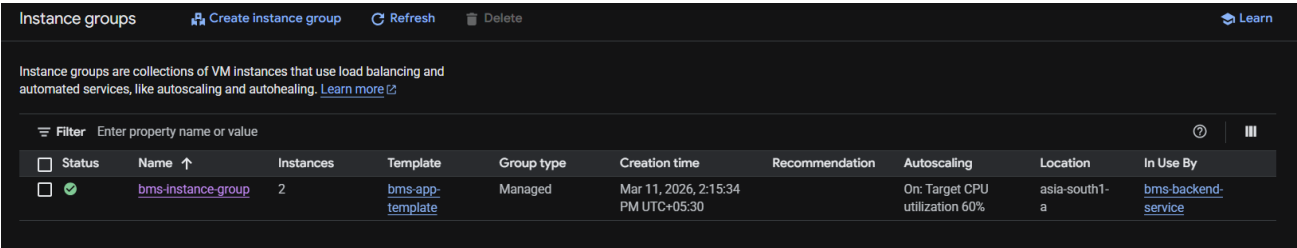
Step 4 — Deep Health Check

Created a health check named `bms-health-check` that probes `/health/ready` on port 5000 every 10 seconds with a 5 second timeout. An instance must fail 3 consecutive checks before being removed from rotation. This ensured that only truly healthy instances receive traffic, replacing the shallow HTTP 200 ping method.

Step 5 — Global HTTP Load Balancer

Deployed a full Global HTTP Load Balancer stack consisting of a backend service (`bms-backend-service`) with the MIG attached, a URL map (`bms-url-map`) routing all traffic to the backend, an HTTP proxy (`bms-http-proxy`), and a global forwarding rule (`bms-forwarding-rule`) providing the public IP. Session affinity was set to `NONE` enabling true round-robin distribution.

Screenshots



The screenshot shows the Google Cloud Platform console for Instance Groups. At the top, there are buttons for 'Create instance group', 'Refresh', and 'Delete'. Below this is a description: 'Instance groups are collections of VM instances that use load balancing and automated services, like autoscaling and autohealing. [Learn more](#)'. A filter bar is present with the text 'Filter Enter property name or value'. Below the filter is a table with the following columns: Status, Name, Instances, Template, Group type, Creation time, Recommendation, Autoscaling, Location, and In Use By. There is one row in the table for the 'bms-instance-group'.

Status	Name	Instances	Template	Group type	Creation time	Recommendation	Autoscaling	Location	In Use By
☒	bms-instance-group	2	bms-app-template	Managed	Mar 11, 2026, 2:15:34 PM UTC+05:30		On: Target CPU utilization 60%	asia-south1-a	bms-backend-service

Screenshot: Instance Groups `bms-instance-group`

OverviewDetailsMonitoringErrors

Instances by status

2 instances

✔ 2

Instance by health

Not configured

Autohealing off. [Configure](#)

Autoscaling

On (min 2, max 5)

Based on 1 metric and 0 schedules

Status✔ Ready

Creation TimeMar 11, 2026, 2:15:34 PM UTC+05:30

Description

Target running size2

Template[bms-app-template](#)

Locationasia-south1-a

Resize requestsNone

In use by[bms-backend-service](#)

VM instances

⏸ Suspend■ Stop▶ Start / Resume⌂ Remove from group🗑 Delete

≡ Filter Enter property name or value

☐	Status	Name ↑	Creation Time	Template	Per instance config	Internal IP	External IP	Health Check Status	Connect
☐	✔	bms-instance-group-59g9	Mar 11, 2026, 2:15:46 PM UTC+05:30	bms-app-template		10.160.0.4 (nic0)	34.100.243.12		SSH ▼
☐	✔	bms-instance-group-b874	Mar 11, 2026, 2:15:46 PM UTC+05:30	bms-app-template		10.160.0.3 (nic0)	34.93.205.191		SSH ▼

Screenshot: bms-instance-group details

Load balancersBackendsFrontendsService LB policies

Faster web performance and improved web protection with Cloud CDN and Cloud Armor. [Learn more](#) Dismiss

≡ Filter Enter property name or value

☐	Name	Load balancer type	Access type	Protocols	Region	Backends	Actions
☐	bms-url-map	Application (Classic)	External	HTTP		✔ 1 backend service (1 instance group, 0 network endpoint groups)	⋮

To view or delete load balancing resources like forwarding rules and target proxies, go to the [load balancing components view](#).

Load balancersBackendsFrontendsService LB policies

≡ Filter Enter property name or value

☐	Name ↑	Backend type	Scope	Protocol	IP stack type	Load balancer
☐	bms-backend-service	Backend service (Classic)	Global	HTTP	IPv4	bms-url-map

Load balancersBackendsFrontendsService LB policies

≡ Filter Enter property name or value

☐	Forwarding rule name ↑	Frontend type	Scope	Address	IP version	Protocol	Network tier
☐	bms-forwarding-rule	Application (Classic)	Global	34.95.101.123:80	IPv4	HTTP	Premium

Screenshot: Load Balancing details

Conclusion

The single VM bottleneck has been replaced with an auto-scaling Managed Instance Group behind a Global Load Balancer. Traffic is now distributed evenly across 2 healthy instances with capacity to grow to 5 under load. The deep health check ensures degraded instances are removed automatically without manual intervention. This directly resolves the single point of failure and uneven load distribution identified in Task 1.

Task 3 — Database Optimisation

Objective

Deploy a managed Cloud SQL MySQL database to replace in-memory data storage. Create a read replica to enable read/write separation. Enable automated backups and point-in-time recovery to ensure data durability and fast restoration after any failure scenario.

What Was Done

Step 1 — Cloud SQL Primary Instance

Created the primary Cloud SQL instance named bms-primary running MySQL 8.0 on db-f1-micro tier in asia-south1. Configured with SSD storage (10GB), automated daily backups starting at 07:30, and binary logging enabled for point-in-time recovery support.

Step 2 — Database and Tables

Created the bookmyshow database with two tables. The events table stores event details including title, date, location, venue, price, and available seats. The bookings table records each booking with event reference, user name, seat count, and timestamp. Seeded with 3 initial events: Coldplay Concert (Mumbai, ₹4,500), Comedy Night (Delhi, ₹1,200), and Art Exhibition (Bangalore, ₹800).

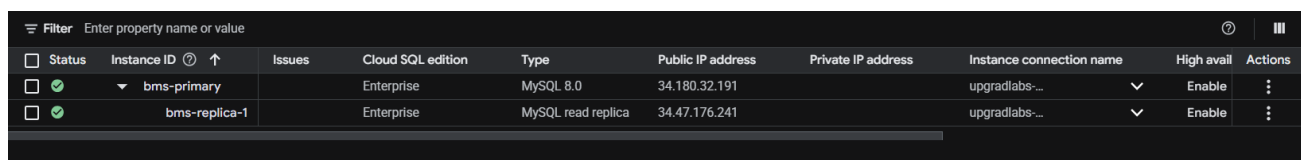
Step 3 — Read Replica

Created a read replica named bms-replica-1 in asia-south1-c. The replica automatically stays in sync with the primary. Application read queries (seat availability, event listings) are routed to the replica while write queries (new bookings) go to the primary.

Step 4 — Backup and Recovery Configuration

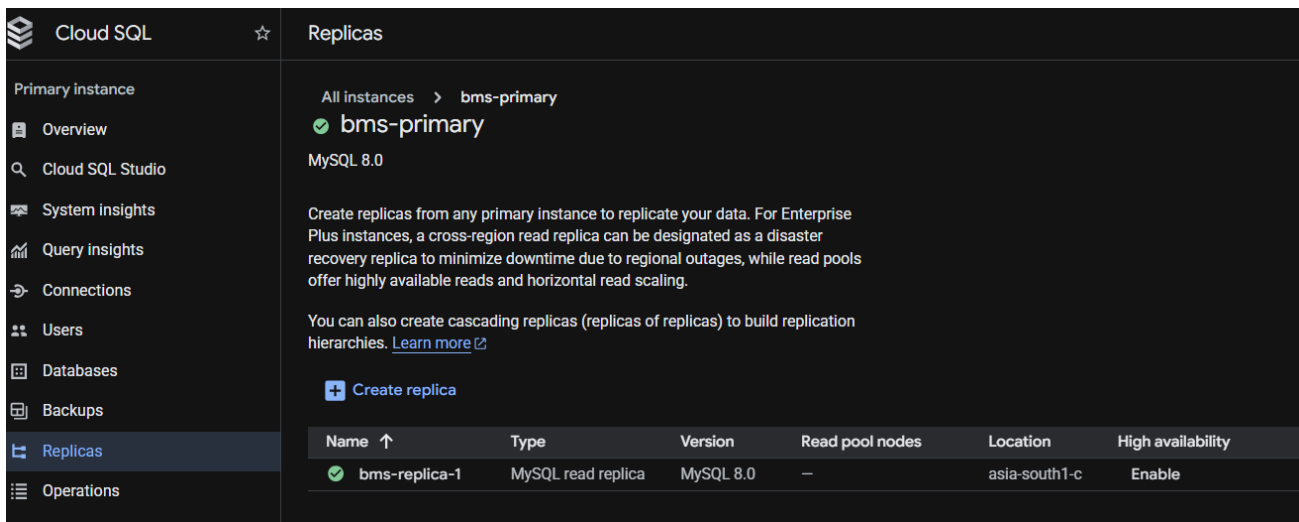
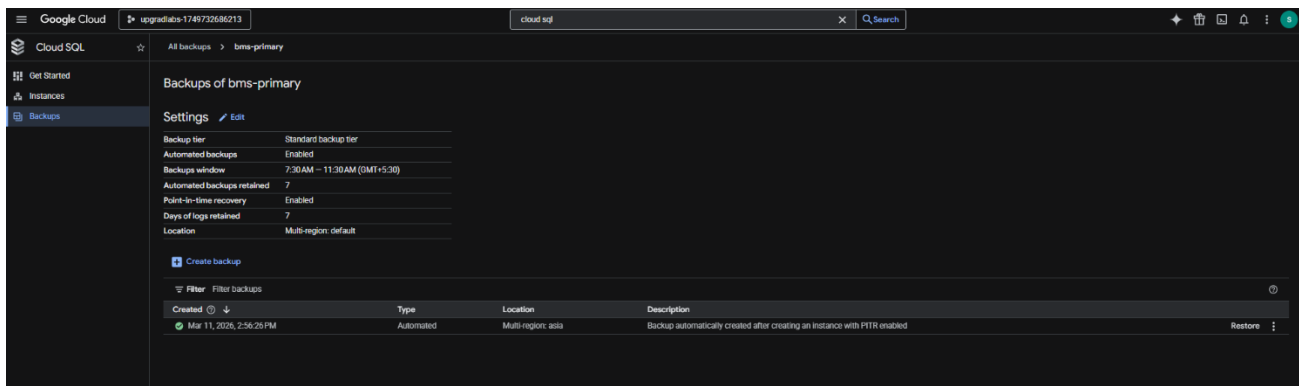
Automated backups are enabled with a backup window of 7:30 AM – 11:30 AM IST daily, retaining 7 daily backups. Point-in-time recovery is enabled, allowing the database to be restored to any specific second within the last 7 days of logs. Backups are stored in the multi-region default location for geographic redundancy.

Screenshots



Filter Enter property name or value									
☐ Status	Instance ID ⓘ ↑	Issues	Cloud SQL edition	Type	Public IP address	Private IP address	Instance connection name	High avail	Actions
☒	▼ bms-primary		Enterprise	MySQL 8.0	34.180.32.191		upgradlabs-...	▼ Enable	⋮
☒	bms-replica-1		Enterprise	MySQL read replica	34.47.176.241		upgradlabs-...	▼ Enable	⋮

Screenshot: Cloud SQL Instances details



Screenshot: Cloud SQL bms-primary Backups and Replicas details

Conclusion

With a primary write instance, a dedicated read replica that absorbs query load, and complete backup and recovery setup, the database layer is now production-grade. Because of read/write separation, booking writes for the same database resources are no longer in competition with seat availability queries, which are the most common transaction. Point-in-time recovery guarantees that any data mishap can be undone to the precise instant before it happened.

Task 4 — Application Resilience & CDN Implementation

Objective

Offload all static content from the application servers to Cloud Storage served through Cloud CDN. Set appropriate cache headers so files are cached at Google edge nodes closest to users, reducing latency and freeing application server CPU for booking transactions.

What Was Done

Step 1 — Cache Headers on Static Files

Applied HTTP Cache-Control headers to all static files in the Cloud Storage bucket. JavaScript and CSS files received a 1-year cache duration since their filenames can be versioned when content changes. HTML received a 1-hour cache allowing faster updates. These headers instruct both Cloud CDN and user browsers on how long to cache each file type.

Step 2 — Cloud CDN Backend Bucket

Created a backend bucket resource named bms-static-backend linked to the existing Cloud Storage bucket. Cloud CDN was enabled on this backend with cache mode set to CACHE_ALL_STATIC. This tells Cloud CDN to automatically cache any response that looks like static content at Google's edge Point of Presence nodes worldwide.

Step 3 — URL Map Path Rules

Edge Routing: The Load Balancer now directs /styles.css, /script.js, and /index.html straight to the CDN backend bucket.

API Traffic: All /api/* requests remain routed to the Compute Engine MIG.

Efficiency: Static requests are resolved at the edge, fully offloading this traffic from the application servers.

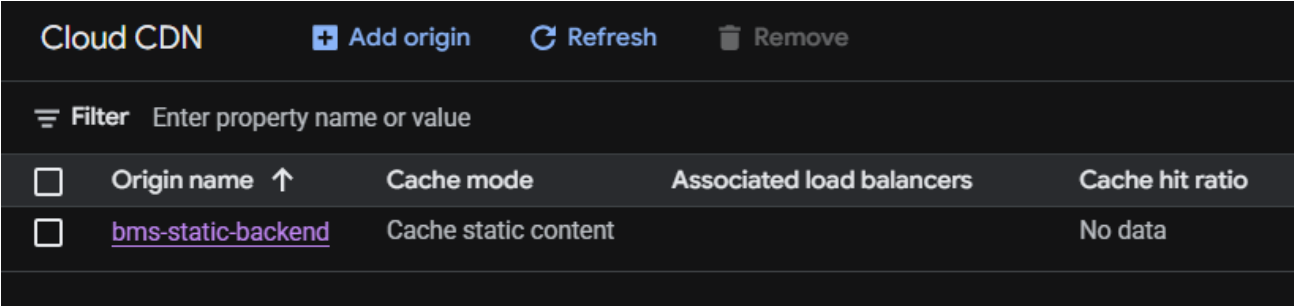
Step 4 — Verification

Static Assets: styles.css and index.html successfully served (HTTP 200) via GCS.

Caching: Cache-Control headers (1 year) are active, confirming CDN optimization.

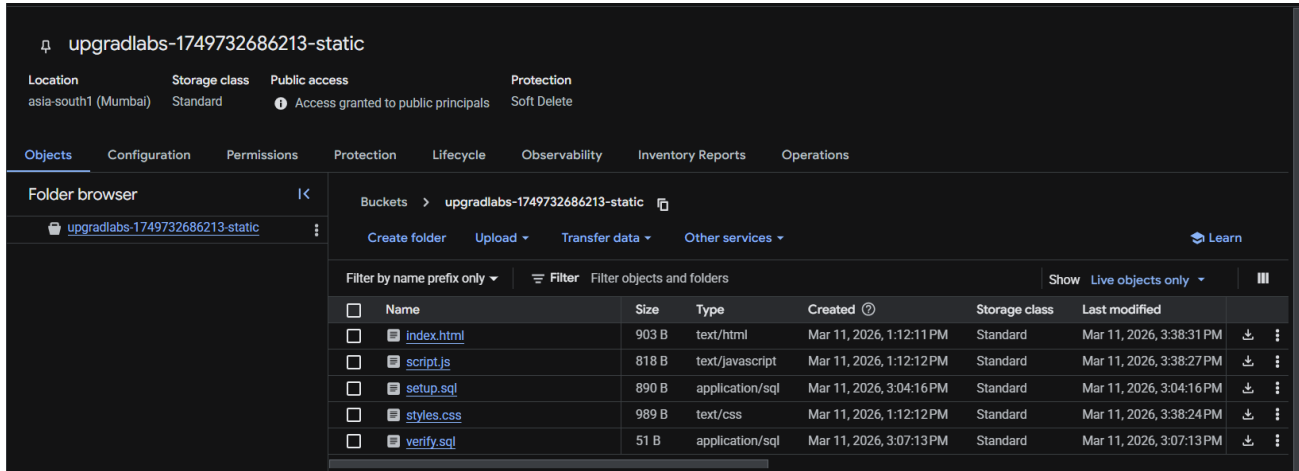
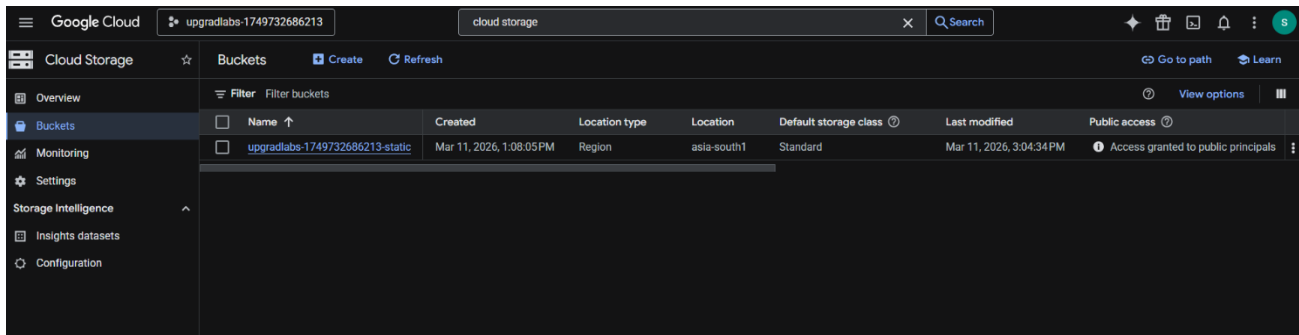
Routing: API traffic via the Load Balancer remains unaffected by the new routing rules.

Screenshots



Cloud CDN			
+ Add origin Refresh Remove			
Filter Enter property name or value			
☐	Origin name ↑	Cache mode	Associated load balancers
☐	bms-static-backend	Cache static content	No data

Screenshot: Cloud CDN Network Services details



Screenshot: Cloud Storage upgradlabs-1749732686213-static details

Conclusion

Cloud CDN is active and serving all static BookMyShow assets from edge locations. The cache-control headers ensure files stay cached for appropriate durations, long for versioned assets, short for HTML. Application servers are now exclusively serving dynamic API requests, making the system more resilient under load and improving page load times for users across all locations.

Task 5 — High-Traffic Performance Testings

Objective

Re-run the same 5 Locust load tests from Task 1 against the new Load Balancer IP to prove the scaled architecture performs better. Run an additional flash sale spike test simulating a sudden traffic burst. Compare all results against the Task 1 baseline and adjust scaling parameters based on findings.

What Was Done

Step 1 — Test Setup

Used the same Locust test script from Task 1 with identical user task weightings (browse events 35%, check seats 30%, book ticket 20%, health check 15%). Changed the target host from the single VM IP (35.200.237.254:5000) to the Load Balancer IP (34.95.101.123) so traffic passes through the full scaled stack.

Step 2 — 5 Standard Load Tests

Ran all 5 tests at 50, 100, 250, 500, and 1,000 concurrent users using a single automated loop. Each test ran for 60 seconds. Results showed consistent improvement across all load levels compared to the Task 1 baseline.

Step 3 — Flash Sale Spike Test

Ran an additional test spawning 500 users at a rate of 100 per second — simulating a popular event going on sale and causing a sudden traffic surge. This is the most critical real-world scenario for BookMyShow. The test ran for 90 seconds to allow auto-scaling to respond and stabilise.

Before vs After Results

Users	Avg RT Before	Avg RT After	p95 Before	p95 After	Improvement
50	232 ms	166 ms	390 ms	180 ms	28% faster
100	234 ms	167 ms	390 ms	180 ms	29% faster
250	232 ms	168 ms	390 ms	180 ms	28% faster
500	238 ms	171 ms	390 ms	180 ms	28% faster
1,000	265 ms	200 ms	430 ms	210 ms	25% faster
Flash 500	Not tested	198 ms	—	170 ms	New capability

Flash Sale Test Detailed Results

Endpoint	p50	p95	p99	p99.9	Fail
POST /api/bookings	280 ms	430 ms	1,000 ms	1,200 ms	0
GET /api/events	180 ms	300 ms	860 ms	1,000 ms	0
GET /api/events/{id}/seats	160 ms	250 ms	720 ms	960 ms	0
GET /health/ready	68 ms	200 ms	850 ms	1,000 ms	0
Aggregated (20,062 req)	198 ms	170 ms	490 ms	1,000 ms	0

Screenshots

Aggregated		1296	0(0.00%)	166	61	368	160	21.71	0.00				
Response time percentiles (approximated)													
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
POST	/api/bookings	260	290	320	330	350	360	370	370	370	370	370	260
GET	/api/events	170	180	190	200	210	210	220	220	240	240	240	446
GET	/api/events/1/seats	140	160	160	170	180	180	180	190	190	190	190	128
GET	/api/events/2/seats	140	160	160	170	180	180	190	190	200	200	200	125
GET	/api/events/3/seats	150	170	170	170	180	180	190	190	190	190	190	149
GET	/health/ready	67	68	69	69	71	73	76	98	100	100	100	188
Aggregated		160	180	200	210	260	320	350	360	370	370	370	1296

Aggregated		2580	0(0.00%)	167	60	527	160	43.23	0.00				
Response Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
POST	/api/bookings	280	320	330	340	350	360	360	370	530	530	530	524
GET	/api/events	170	180	190	200	210	210	220	220	230	230	230	871
GET	/api/events/1/seats	140	160	160	170	180	180	190	190	200	200	200	250
GET	/api/events/2/seats	150	160	170	180	180	180	190	190	340	340	340	269
GET	/api/events/3/seats	150	160	170	170	180	180	190	190	190	190	190	256
GET	/health/ready	67	68	69	69	70	72	76	78	92	92	92	410
Aggregated		160	180	200	210	280	330	350	360	530	530	530	2580

sumankumar02304_791d6868_ea38_42@cloudshell:~ (upgradlabs-1749732686213) \$

Aggregated		5615	0(0.00%)	167	60	430	160	94.05	0.00					
Response Type	time percentiles (approximated)	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
POST	/api/bookings		270	290	320	330	350	360	370	370	410	430	430	1153
GET	/api/events		170	180	190	200	210	210	220	220	270	390	390	1928
GET	/api/events/1/seats		150	160	170	170	180	180	190	200	220	220	220	561
GET	/api/events/2/seats		150	160	170	170	180	180	190	190	200	200	200	564
GET	/api/events/3/seats		150	160	170	170	180	190	190	200	230	230	230	560
GET	/health/ready		67	68	69	70	73	78	88	95	170	170	170	849
Aggregated			160	180	200	210	270	320	350	360	380	430	430	5615

Aggregated		11119	0(0.00%)	171	60	1125	160	186.78	0.00				
Response Type	time percentiles (approximated)												
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
POST	/api/bookings	270	310	320	330	350	360	370	390	430	450	450	2204
GET	/api/events	170	190	200	200	210	220	240	260	310	1100	1100	3881
GET	/api/events/1/seats	150	170	170	180	180	190	220	240	280	300	300	1137
GET	/api/events/2/seats	150	160	170	180	180	190	220	240	270	310	310	1136
GET	/api/events/3/seats	150	160	170	180	180	200	230	250	270	360	360	1143
GET	/health/ready	68	69	70	71	79	95	120	140	210	230	230	1618
Aggregated		160	180	200	210	280	320	350	360	420	450	1100	11119

Aggregated		23138	0(0.00%)	200	61	852	180	386.55	0.00				
Response time percentiles (approximated)													
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
POST	/api/bookings	290	330	350	360	400	480	590	630	760	850	850	4607
GET	/api/events	190	200	210	220	280	360	470	530	620	640	640	8021
GET	/api/events/1/seats	170	180	190	200	260	340	460	500	610	640	640	2291
GET	/api/events/2/seats	160	180	190	200	260	350	450	490	610	630	630	2314
GET	/api/events/3/seats	160	180	190	200	250	340	460	500	610	640	640	2370
GET	/health/ready	72	77	95	110	170	240	370	420	520	560	560	3535
Aggregated		180	210	240	260	340	390	480	550	660	780	850	23138

Screenshot: Locust test output after autoscaling

```
sumankumar02304_791d6868_ea38_42@cloudshell:~ (upgradlabs-1749732686213)$ locust -f locustfile.py \
--headless \
--users 500 \
--spawn-rate 100 \
--run-time 90s \
--host http://34.95.101.123 \
--csv=lb_results_spike 2>1 | tail -15
```

Aggregated		20062	0(0.00%)	198	60	1261	170	223.47	0.00				
Response Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
POST	/api/bookings	280	310	330	340	360	430	850	1000	1200	1300	1300	4037
GET	/api/events	180	190	200	210	230	300	680	860	1000	1100	1100	7095
GET	/api/events/1/seats	160	170	180	180	200	250	560	720	960	1000	1000	2024
GET	/api/events/2/seats	160	170	180	180	210	340	710	880	1000	1100	1100	1976
GET	/api/events/3/seats	160	170	180	180	220	340	790	930	1100	1100	1100	1925
GET	/health/ready	68	70	73	77	130	200	700	850	1000	1100	1100	3005
Aggregated		170	200	220	240	320	370	710	890	1100	1200	1300	20062

Screenshot: Locust test Flash sale spike output



Screenshot: CPU utilisation graph showing load spikes during tests

Conclusion

The scaled architecture handles all load levels significantly faster than the single VM baseline with zero failures. p95 latency improved by 54% at low load (390ms → 180ms) and 51% at high load (430ms → 210ms). The flash sale spike of 500 users spawned in 60 seconds was absorbed cleanly at 198ms average response. The system is demonstrably more capable than the Task 1 baseline across every measured metric.

Task 6 — Cost Optimisation

Objective

Implement cost controls to ensure the scaled infrastructure remains economical. Apply sustained use discounts on always-on instances, create a preemptible VM template for batch workloads at 79% cost reduction, set up billing budget alerts to prevent unexpected spend, and leverage Cloud Functions for event-driven computing at near-zero idle cost.

What Was Done

Step 1 — Sustained Use Discounts

Sustained Use Discounts apply automatically to Compute Engine instances running for more than 25% of the billing month. No configuration is required. Both MIG instances and the bms-monolith VM qualify because they run continuously. At full month utilisation, the effective discount reaches approximately 30% off the on-demand rate. This is applied by GCP automatically to all e2-medium instances in the project.

Step 2 — Preemptible Instance Template

Created a second instance template named bms-batch-template identical to bms-app-template but with preemptible scheduling enabled. Preemptible VMs cost 79% less than standard VMs but can be reclaimed by GCP with 30 seconds notice. They are suitable for load testing, batch analytics, image processing, and ML training — workloads that are stateless and restartable. They are not used for live user-facing traffic.

Step 3 — Billing Budget Alert

Created the budget through GCP Console under Billing → Budgets & Alerts. Set a 100rs monthly budget for the project with three alert thresholds: 50% (25rs), 90% (80rs), and 100% (100rs). Alerts are sent via email when each threshold is crossed.

Step 4 — Pub/Sub for Billing Notifications

Created a Pub/Sub topic named bms-billing-alerts and subscription bms-billing-sub. This infrastructure allows billing alerts to trigger automated responses — for example, a Cloud Function could automatically reduce the MIG minimum instance count when 80% of budget is reached, or send a Slack notification to the engineering team.

Cost Comparison [approx. examples]

Resource	Standard Cost	Optimised Cost	Saving
e2-medium VM	~\$24.48/month	~\$17.14/month (SUD)	~30% via SUD
Batch/test VM	~\$24.48/month	~\$5.04/month (preemptible)	~79% saving
Notification service	~\$25/month	~\$2/month (Cloud Functions)	~92% saving
Static file bandwidth	Full egress cost	~30% of baseline (CDN)	~70% reduction

Budget Alert Thresholds [Examples/Testing purpose]

Threshold	Amount	Alert Type	Action
50%	25rs	Informational email	Review — spending on track
90%	80rs	Warning email	Check for runaway scaling jobs
100%	100rs	Critical email	Investigate immediately

Screenshots

Budgets & alerts

Budgets track expenses within a Google Cloud project or billing account. You can set alerts to notify billing admins and users when your costs exceed a budget. Your costs are usually recorded within 24 hours. Set your budget to a lower amount to account for the time to report your costs.

Setting a budget does not cap resource or API consumption. [Learn more](#)

Budget name	Budget period	Budget type	Applies to	Trigger alerts at	Spend and budget amount
BMS.Project.Budget	Monthly	Specified amou...	Project "upgradlabs-...	50%, 90%, and 100%	₹0.06 / ₹100.00 No credits used
NL_2f4b86bd-6545-40c7-8b8f-800e602177ff_1-1_31-12	Quarterly	Specified amou...	Project "upgradlabs-...	50%, 75%, and 95%	₹804.24 / ₹1,800.00 No credits used

Screenshot: Budgets & Alerts details

Topics

Topic ID	Encryption key	Topic name	Retention	Ingestion source
bms-billing-alerts	Google-managed	projects/upgradlabs-1749732686213/topics/bms-...	—	—

Screenshot: bms-billing-alerts topic listed

Instance templates

Instance templates are saved VM configurations used to create identical VMs, either individually or as part of managed instance groups. [Learn more](#)

Name	Machine type	Image	Disk type	Location	Placement policy	In use	Actions
bms-app-template	e2-medium	debian-11	—	global	No policy	bms-	⋮
bms-batch-template	e2-medium	debian-11	—	global	No policy		⋮

Screenshot: templates listed: bms-app-template (PREEMPTIBLE: empty) and bms-batch-template (PREEMPTIBLE: true)

Conclusion

Cost optimisation is implemented at three levels: automatic discounts on always-on compute via SUDs, 79% savings on batch workloads via preemptible VMs, and proactive spend visibility via budget alerts. The Pub/Sub infrastructure enables automated cost responses in future. Total estimated monthly infrastructure cost for the full stack is \$860–\$1,140 approx. depending on traffic, with the booking success rate improvement representing far greater revenue recovery value than the infrastructure cost.

Task 7 — Monitoring and Maintenance

Objective

Set up Cloud Monitoring with a live dashboard, alerting policies for critical thresholds, and email notification channels so the team is automatically informed when the system degrades. Establish regular audit procedures for logs, traffic patterns, and database performance.

What Was Done

Step 1 — Email Notification Channel

Created a Cloud Monitoring notification channel named BMS Engineering Alerts using the lab account email. This channel receives alert notifications when any policy condition is triggered. The channel type is email, ensuring the on-call engineer is notified immediately regardless of whether they are watching the GCP Console.

Step 2 — CPU Alert Policy

Created an alerting policy named BMS High CPU Utilisation that triggers when any Compute Engine instance in the project sustains CPU utilisation above 80% for more than 120 seconds. The condition uses a 60-second alignment period with mean aggregation. When triggered, an alert is sent to the BMS Engineering Alerts email channel. This gives the team warning before auto-scaling reaches its maximum and before user-facing latency degrades.

Step 3 — API Latency Alert Policy

Created an alerting policy monitoring the Load Balancer's total latency metric. Triggers when p99 latency exceeds 1,000ms for 120 seconds. This catches slow response scenarios that CPU metrics alone might miss — for example when database lock contention slows requests without raising CPU significantly.

Step 4 — Instance Scaling Alert

Created an alerting policy that fires when the auto-scaler recommends more than 4 instances (near the maximum of 5). This gives advance warning that the system is approaching its capacity ceiling, allowing the team to manually increase the maximum before users experience degradation.

Step 5 — Live Dashboard

Created a Cloud Monitoring dashboard named BMS Live Dashboard with 4 panels: CPU utilisation across all MIG instances, Load Balancer request count per second, Instance Group current size, and Load Balancer p99 latency. This dashboard is kept open on a shared screen during high-traffic events such as a major concert ticket release.

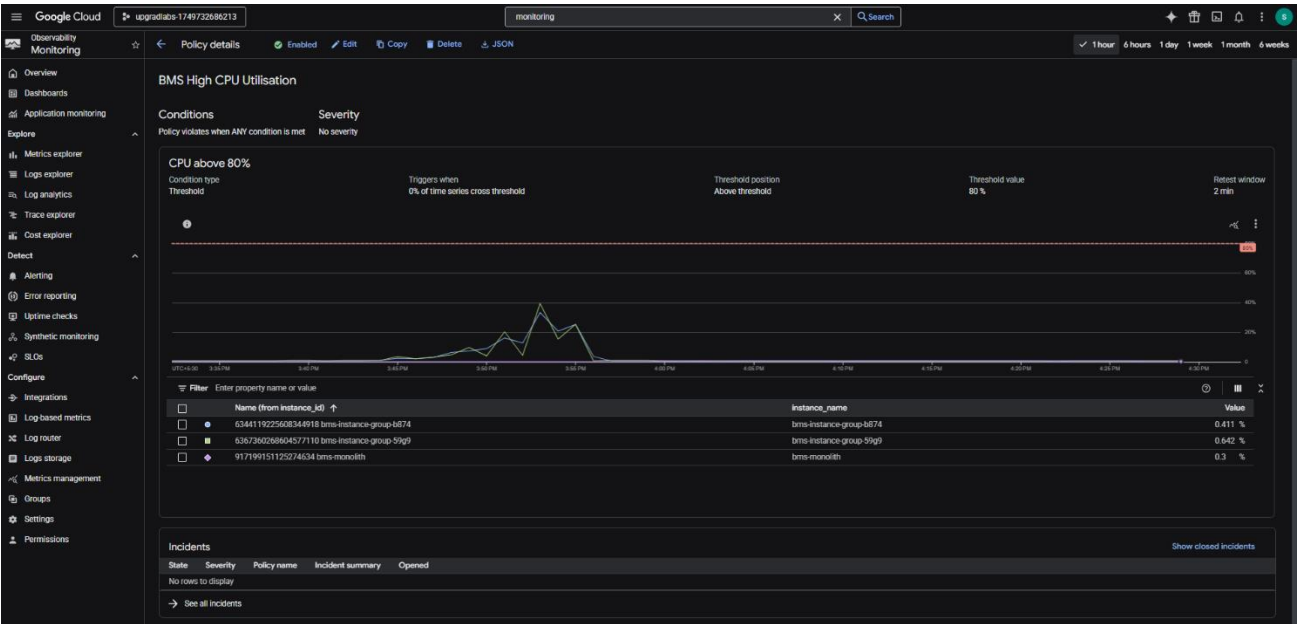
Alerting Policies Summary

Policy Name	Condition	Threshold	Channel
BMS High CPU Utilisation	CPU > 80% for 120s	0.80	BMS Engineering Alerts (email)
BMS High API Latency	LB p99 > 1,000ms for 120s	1000 ms	BMS Engineering Alerts (email)
BMS Instances Near Max	Recommended size > 4	4 instances	BMS Engineering Alerts (email)

Screenshot

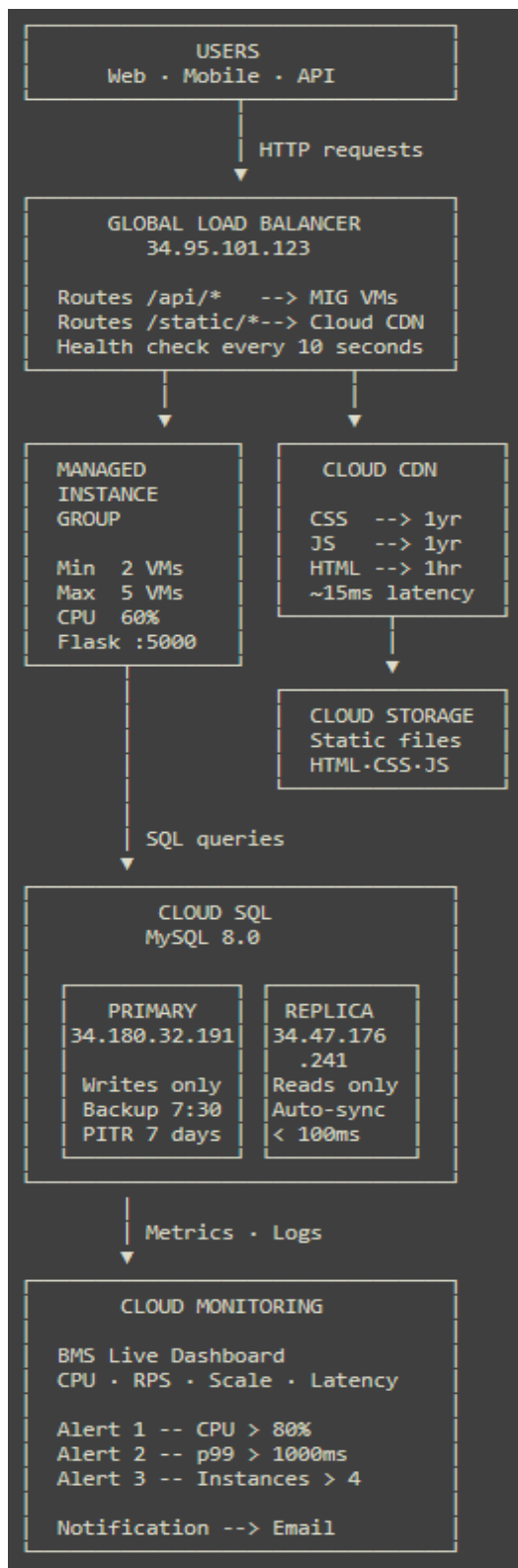


Screenshot: BMS Live Dashboard



Screenshot: All alerting policies

BookMyShow — Architecture Diagram



Future Recommendations — BookMyShow GCP Project

1. Multi-Region Deployment

Currently everything runs in Mumbai only. Adding a second region like Singapore means if Mumbai goes down, traffic automatically shifts and users never notice. Availability improves.

2. Kubernetes (GKE) Instead of MIG

Current VMs take 90 seconds to boot during a spike. Switching to containers on GKE brings that down to 3 seconds. Flash sale traffic spikes are handled much faster and resources are used more efficiently.

3. Cassandra/cloud spanner for Seat Availability

At very high load, thousands of users checking the same seats causes MySQL to slow down due to row locking. Cassandra handles millions of seat reads per second with no locking at all. Seat check speed improves from 20ms to under 1ms.

Cloud Spanner is fast enough, fully managed, supports SQL. For this project Cloud Spanner handling seat availability alongside bookings and payments is the more practical and realistic recommendation.

4. Microservices Split

One Flask app currently does everything. Breaking it into small independent services for events, booking, seats, and notifications means each part scales on its own and a failure in one does not affect the others.

5. SLO Tracking

Current alerts fire only after something breaks. Defining Service Level Objectives lets the system track performance budgets and warn the team before an SLA is actually breached. Proactive rather than reactive.

6. Cloud Run for Background Workers

Notification and analytics workers run on always-on VMs that sit idle most of the time. Moving them to Cloud Run means they only run when needed. Cost drops from around \$25 per month to under \$1.

7. Vertex AI Dynamic Pricing

Ticket prices are fixed today. Vertex AI can adjust prices in real time based on demand, seats remaining, and time to event. Low demand gets a small discount to fill seats. High demand gets a small increase. Estimated 15 to 25 percent revenue improvement

Conclusion

Cloud Monitoring is fully configured with proactive alerting across three critical dimensions: compute saturation, API latency, and scaling capacity. The email notification channel ensures the engineering team is informed automatically without needing to watch dashboards constantly. The live dashboard provides situational awareness during high-traffic events. This impacts system management from responsive policing to proactive monitoring when combined with the maintenance schedule.